

Assignment Sheet 1

Problem 0

On <https://fmi.uni-stuttgart.de/alg/research/stuff/> you can find several sample graphs to download. They adhere to the following format:

1. number of nodes
2. number of edges
3. nodes
4. edges

A node is defined by:

`<node ID> <OSM ID> <longitude> <latitude> <elevation>`

An edge is defined by:

`<start node ID> <end node ID> <weight> <type> <maxspeed>`

The node

`0 1212985 53.822330400000000557 10.723397400000000141 0`

is a node with

- ID 0
- Osm Id 1212985
- longitude 53.822330400000000557
- latitude 10.723397400000000141
- an elevation of 0

The edge

`626656 626655 67 15 30`

is an edge

- from node 626656 to node 626655
- with weight/cost of 67
- of type 15 = highway:service
- with a max speed of 30 km/h

Graph files can also be created using the OsmGraphCreator (<https://github.com/fmi-alg/OsmGraphCreator>). To that end you have to invoke the program as follows:

```
./creator -g fmimaxspeedtext -t distance -c ../../data/configs/all.cfg \  
-o output.graph -s source.osm.pbf
```

Raw OSM data can be found on <http://download.geofabrik.de/>.

Problem 1

Implement a program which reads in such graphs and stores them in memory (suggested data structure: offset arrays).

Important: Your graph representation should allow for iterating over both incoming as well as outgoing edges.

Problem 2

Determine the number of weakly connected components in the graph that has been read and measure the running time. For the German road network, running time (excluding file input) must be below 10s.

Problem 3

Investigate the effect of changing the order of nodes; permute the nodes randomly and repeat the previous experiment.

Problem 4

Implement Dijkstra's algorithm for computing shortest paths. Choose random source and target nodes in the graph and compute the shortest path. Measure running time for 100 source target pairs. For the German road network, the query times should be less than 10s on average.

Problem 5

Extend your implementation such that it can read a file with source target pairs for which distances are then computed (and written out to a file). Verify your distances using the distance tables provided on <https://fmi.uni-stuttgart.de/alg/research/stuff/>. The input file should have the following format per line:

```
<Start ID> <Target ID>
```

The output file should have the following format:

```
<Start ID> <Target ID> <distance> <query time>
```

Problem 6

Submit your program code together with a README how to build/run your code. Note that we will test your program automatically, so your program has to run without further changes and produce the output as defined above.

More concretely, for this assignment we must be able to execute your code as follows:

- unpack your code from a `.tar.bz2` file
- *we* copy files `graph.fmi` and `queries.txt` into the top level directory of your code package
- we type `make`; the following should happen:
 - your code reads `graph.fmi`, computes the number of weakly connected components and outputs this number as well as the running time (problem 2)
 - your code reads `graph.fmi`, `queries.txt` and writes the results into a file `result.txt` (problem 5)